

Uni-App 首页调用网络接口实现真实数据渲染

在Uni-App开发中，首页作为应用入口，展示真实有效的数据是核心需求。本文将选用公开免费的电商类API，完整实现“接口请求-数据解析-页面渲染-异常处理”全流程，同时融入加载状态、空数据提示等用户体验优化，确保首页数据展示既真实又稳定。

一、前期准备：接口选型与基础配置

选择合适的公开接口是实现真实数据渲染的前提，需优先考虑接口稳定性、数据格式规范性及无跨域限制（或支持跨域）。本文选用**FastMock模拟的电商首页接口**，该接口支持GET请求，返回轮播图、分类、推荐商品等标准电商数据。

1. 接口信息说明

接口类型	详细信息
请求地址	https://www.fastmock.site/mock/1f177d8818a527707d30a790e126411d/api/home/index
请求方式	GET
请求头	Content-Type: application/json
返回数据格式	包含swiper（轮播图）、category（分类）、recommend（推荐商品）三个数据数组

2. 跨域配置（关键）

Uni-App开发中，本地调试网络接口可能出现跨域问题，需在项目根目录的`manifest.json`中配置跨域白名单（仅H5端生效），或在HBuilderX中开启“内置浏览器跨域”功能。

Code block

```
1  // manifest.json -> H5配置
2  "h5": {
3    "devServer": {
4      "proxy": {
```

```

5      "/api": {
6        "target": "https://www.fastmock.site",
7        "changeOrigin": true,
8        "pathRewrite": {
9          "^/api": ""
10       }
11     }
12   }
13 }
14 }

```



若使用真机调试或云打包，跨域配置无需额外处理，Uni-App会自动代理请求；仅本地H5调试时需配置上述代理。

3. 封装网络请求工具

为提升代码复用性与可维护性，先封装统一的网络请求工具，包含请求拦截、响应拦截、异常处理等功能，避免在页面中重复编写请求逻辑。

Code block

```

1  // utils/request.js
2  /**
3   * 统一网络请求工具
4   * @param {Object} options - 请求配置
5   * @returns {Promise} 请求结果
6   */
7  export const request = (options) => {
8    // 基础配置
9    const baseUrl = ''; // 若配置代理，此处可留空；否则填写完整接口域名
10   const defaultOptions = {
11     url: baseUrl + options.url,
12     method: options.method || 'GET',
13     header: {
14       'Content-Type': 'application/json',
15       ...options.header
16     },
17     data: options.data || {}
18   };
19
20   // 显示加载中提示（默认显示，可通过options.hideLoading关闭）
21   if (!options.hideLoading) {
22     uni.showLoading({

```

```
23     title: '加载中...',
24     mask: true
25   });
26 }
27
28 return new Promise((resolve, reject) => {
29   // 发起请求
30   uni.request({
31     ...defaultOptions,
32     // 请求成功
33     success: (res) => {
34       // 关闭加载提示
35       if (!options.hideLoading) {
36         uni.hideLoading();
37       }
38
39       // 接口返回格式: { code: 200, data: {}, msg: '' }
40       const { code, data, msg } = res.data;
41       if (code === 200) {
42         // 业务成功, 返回数据
43         resolve(data);
44       } else {
45         // 业务失败, 提示错误信息
46         uni.showToast({
47           title: msg || '接口请求失败',
48           icon: 'none',
49           duration: 2000
50         });
51         reject(new Error(msg || '业务请求失败'));
52       }
53     },
54     // 请求失败 (网络错误、超时等)
55     fail: (err) => {
56       // 关闭加载提示
57       if (!options.hideLoading) {
58         uni.hideLoading();
59       }
60
61       // 提示网络错误
62       uni.showToast({
63         title: '网络异常, 请稍后重试',
64         icon: 'none',
65         duration: 2000
66       });
67       reject(err);
68     },
69     // 请求完成 (无论成功失败)
```

```

70         complete: () => {
71             // 可添加请求完成后的统一逻辑（如关闭加载动画）
72         }
73     });
74 });
75 };
76
77 // 简化GET请求
78 export const get = (url, data = {}, options = {}) => {
79     return request({
80         url,
81         method: 'GET',
82         data,
83         ...options
84     });
85 };
86
87 // 简化POST请求
88 export const post = (url, data = {}, options = {}) => {
89     return request({
90         url,
91         method: 'POST',
92         data,
93         ...options
94     });
95 };

```

4. 封装首页API请求方法

在API模块中单独封装首页数据请求方法，与页面逻辑分离，便于后续维护。

Code block

```

1  // api/home.js
2  import { get } from '@/utils/request';
3
4  /**
5   * 获取首页数据（轮播图、分类、推荐商品）
6   * @returns {Promise} 首页数据
7   */
8  export const getHomeData = () => {
9      return get('/mock/1f177d8818a527707d30a790e126411d/api/home/index');
10 };

```

二、首页实现：数据请求与真实渲染

基于前文封装的布局组件与网络工具，在首页中调用API获取真实数据，实现轮播图、分类入口、推荐商品列表的动态渲染，并处理加载中、空数据、异常等边界情况。

1. 首页完整代码（index.vue）

Code block

```
1  <template>
2    <page-frame
3      page-title="首页"
4      :enable-pull-down="true"
5      @pull-down="onPullDownRefresh"
6      :is-loading-more="isLoadingMore"
7      @reach-bottom="onReachBottom"
8    >
9
10   <view class="home-swiper" v-if="swiperList.length > 0">
11     <uni-swiper
12       :autoplay="true"
13       :interval="3000"
14       :duration="500"
15       :indicator-dots="true"
16       :indicator-active-color="#FF4444"
17     >
18       <uni-swiper-item v-for="(item, index) in swiperList" :key="index">
19         <image
20           :src="item.image"
21           mode="widthFix"
22           class="swiper-img"
23           @click="handleSwiperClick(item.link)"
24         ></image>
25       </uni-swiper-item>
26     </uni-swiper>
27   </view>
28
29
30   <view class="category-container" v-if="categoryList.length > 0">
31     <view class="container-title">
32       <text class="title-text">全部分类</text>
33       <text class="more-text" @click="toCategoryPage">更多 </text>
34     </view>
35     <view class="category-list">
36       <view
37         class="category-item"
38         v-for="(item, index) in categoryList"
39         :key="index"
40         @click="toGoodsListByCategory(item.id, item.name)"
41       >
42         <image :src="item.icon" mode="widthFix" class="category-icon">
43
44         <text class="category-name">{{ item.name }}</text>
45       </view>
46     </view>
47   </view>
```

```

47
48
49     <view class="recommend-container">
50         <view class="container-title">
51             <text class="title-text">为你推荐</text>
52             <text class="more-text" @click="toAllRecommend">更多 ></text>
53         </view>
54
55
56         <view class="goods-list" v-if="recommendList.length > 0">
57             <view
58                 class="goods-item"
59                 v-for="(item, index) in recommendList"
60                 :key="item.id"
61                 @click="toGoodsDetail(item.id)"
62             >
63                 <view class="goods-img-wrap">
64                     <image :src="item.cover" mode="widthFix" class="goods-img"></image>
65                     <view class="goods-tag" v-if="item.tag">{{ item.tag }}</view>
66                 </view>
67                 <text class="goods-name">{{ item.name }}</text>
68                 <view class="goods-price-wrap">
69                     <text class="goods-price">¥{{ item.price.toFixed(2) }}</text>
70                     <text class="goods-sales">{{ item.sales }}人已购</text>
71                 </view>
72             </view>
73         </view>
74
75
76         <view class="empty-tips" v-else-if="!isLoading">
77             <image src="/static/empty-goods.png" mode="widthFix" class="empty-img">
78             </image>
79             <text class="empty-text">暂无推荐商品</text>
80         </view>
81
82
83         <view class="global-empty" v-if="isError">
84             <image src="/static/error-network.png" mode="widthFix" class="error-img">
85             </image>

```

2. 核心逻辑解析

(1) 数据请求流程

1. **状态管理**：通过`isLoading`（加载中）、`isError`（异常）、`isLoadingMore`（上拉加载）三个状态，精准控制页面UI展示，提升用户体验；

2. **异步处理**：使用`async/await`简化异步请求逻辑，相比回调函数更易读、易维护；
3. **异常捕获**：通过`try/catch`捕获API请求中的异常（网络错误、业务错误），并展示“重新加载”按钮，提供补救措施。

(2) 数据渲染优化

1. **key值优化**：推荐商品列表使用商品唯一ID（`item.id`）作为`v-for`的key，而非索引，提升Vue渲染性能；
2. **条件渲染**：根据数据状态展示不同UI（加载中→真实数据→空数据→异常提示），避免页面空白或错乱；
3. **文本处理**：商品名称使用`-webkit-line-clamp`限制显示行数，避免文本溢出影响布局。

(3) 分页加载实现

1. **分页参数**：通过`pageNum`（页码）和`pageSize`（每页条数）控制分页请求；
2. **数据处理**：首次加载（`pageNum=1`）时覆盖商品列表，分页加载时追加数据（`[...this.recommendList, ...newData]`）；
3. **防重复加载**：上拉加载前判断当前状态，避免同时发起多个加载请求。

三、关键优化点：提升数据渲染体验

1. 加载状态优化

- **全局加载与局部加载结合**：首页整体数据使用全局加载提示，列表项可添加局部骨架屏（Skeleton），减少等待焦虑；
- **加载超时处理**：在请求工具中添加超时配置，超时后触发重试逻辑：

```
// utils/request.js 中添加超时配置
const defaultOptions = {
  timeout: 10000, // 10秒超时
  ...
};
```

2. 数据缓存优化

首页数据可缓存到本地，下次打开页面时先展示缓存数据，再请求最新数据，提升首屏加载速度：

Code block

```
1  async getHomeData() {
2    this.isLoading = true;
3    this.isError = false;
4
5    // 1. 先读取本地缓存数据
```

```

6    const cacheHomeData = uni.getStorageSync('homeData');
7    if (cacheHomeData && this.pageNum === 1) {
8        // 展示缓存数据
9        this.swiperList = cacheHomeData.swiper || [];
10       this.categoryList = cacheHomeData.category || [];
11       this.recommendList = cacheHomeData.recommend || [];
12   }
13
14   try {
15       // 2. 请求最新数据
16       const homeData = await getHomeData();
17
18       // 3. 更新页面数据
19       this.swiperList = homeData.swiper || [];
20       this.categoryList = homeData.category || [];
21       this.recommendList = this.pageNum === 1 ? homeData.recommend :
[...this.recommendList, ...homeData.recommend];
22
23       // 4. 缓存最新数据 (设置10分钟过期)
24       uni.setStorageSync('homeData', homeData);
25       uni.setStorageSync('homeDataExpire', Date.now() + 10 * 60 * 1000);
26   } catch (err) {
27       // 若缓存存在且未过期, 不显示错误提示
28       const expireTime = uni.getStorageSync('homeDataExpire');
29       if (!cacheHomeData || Date.now() > expireTime) {
30           this.isError = true;
31       }
32   } finally {
33       this.isLoading = false;
34       this.isLoadingMore = false;
35   }
36   }

```

3. 图片优化

图片是首页加载的性能瓶颈, 需从加载速度与显示效果两方面优化:

- **图片懒加载:** 为商品图片添加`lazy-load`属性, 实现滚动到可视区域再加载:

```
<image :src="item.cover" mode="widthFix" class="goods-img" lazy-load></image>
```

- **图片压缩与格式:** 建议接口返回不同尺寸的图片 (如缩略图、原图), 首页使用缩略图; 优先使用 WebP 格式, 减小图片体积;

- **占位图与错误图：**为图片添加默认占位图和错误占位图，避免加载过程中页面错乱：

```
<image
  :src="item.cover"
  mode="widthFix"
  class="goods-img"
  lazy-load
  :placeholder="true"
  placeholder-src="/static/placeholder.png"
  @error="e => e.target.src = '/static/error-img.png'"
></image>
```

4. 性能优化：减少渲染阻塞

- **数据解构优化：**避免在模板中直接使用复杂数据表达式（如`item.goodsInfo.name`），提前在脚本中解构数据；
- **避免频繁数据修改：**分页加载时使用数组展开运算符（`[...arr1, ...arr2]`）创建新数组，而非`push`修改原数组，减少Vue响应式触发次数；
- **组件懒加载：**对于非首屏必要的组件（如弹窗、详情页组件），使用`import()`实现懒加载，减少首屏JS体积：

```
// 懒加载组件
const GoodsDetail = () => import('@pages/goods/GoodsDetail.vue');
```

四、常见问题与解决方案

1. 问题1：接口请求无响应或跨域报错

现象：本地调试时，接口请求一直处于加载中，或控制台报“Access to XMLHttpRequest at ... from origin ... has been blocked by CORS policy”错误。

解决方案：

1. 检查`manifest.json`中的H5跨域代理配置是否正确，确保`target`指向接口域名；
2. 在HBuilderX中开启“运行→运行到浏览器→启用跨域代理”功能；
3. 若接口本身支持跨域（返回`Access-Control-Allow-Origin`头），可直接删除代理配置，使用完整接口地址请求。

2. 问题2：数据渲染后页面布局错乱

现象：接口数据返回后，轮播图高度异常、商品列表排版错乱。

解决方案：

1. 为图片设置固定宽高比（如轮播图`aspect-ratio: 750/300`），避免图片加载完成后挤压布局；

2. 使用`box-sizing: border-box`统一盒模型，避免padding和border导致宽度溢出；
3. 商品列表使用Flex布局时，添加`align-items: flex-start`，避免因商品信息高度不一致导致对齐异常。

3. 问题3：分页加载时数据重复或漏加载

现象：上拉加载更多时，出现重复商品数据，或部分商品数据未加载。

解决方案：

1. 检查分页参数传递是否正确，确保`pageNum`每次上拉时自增1；
2. 与后端确认接口返回的分页数据是否正确（如`pageNum=2`时返回第11-20条数据）；
3. 添加数据去重逻辑，基于商品ID过滤重复数据：

```
// 分页加载时去重
```

```
const newData = homeData.recommend || [];  
const uniqueData = newData.filter(item =>  
!this.recommendList.some(prev => prev.id === item.id));  
this.recommendList = [...this.recommendList, ...uniqueData];
```

4. 问题4：下拉刷新后数据不更新

现象：触发下拉刷新后，页面数据仍为旧数据，未加载最新内容。

解决方案：

1. 检查下拉刷新回调中是否重置`pageNum=1`，确保请求第一页最新数据；
2. 确认`uni.stopPullDownRefresh()`是否在数据加载完成后调用，避免刷新动画未结束导致页面状态异常；
3. 清除本地缓存数据（若有），避免缓存数据覆盖最新请求结果。

五、总结：首页真实数据渲染的核心要点

Uni-App首页实现真实数据渲染，核心是“稳定请求+高效渲染+优质体验”，需把握以下要点：

1. **接口层：**封装统一请求工具，处理异常、加载状态与跨域问题，确保数据请求稳定；
2. **数据层：**解析接口数据并合理管理状态（加载中、空数据、异常），结合分页与缓存优化数据流转；
3. **渲染层：**使用条件渲染、合理设置key、优化图片加载，确保页面布局稳定、渲染高效；
4. **体验层：**添加加载提示、空数据提示、异常重试按钮，提升用户在各种场景下的体验。



实际开发中，需根据接口返回的真实数据结构调整数据解析逻辑，同时结合应用场景（如电商、资讯、工具）优化页面布局与交互，确保数据展示既准确又符合用户习惯。

(注：文档部分内容可能由 AI 生成)